

Projeto Final Integrado: Pipeline de Dados em Nuvem para Aprendizagem de Máquina

*Reconhecimento de Atividade Humana com Sensores de Smartwatch
(WISDM)*

Igor Coutinho Ferreira Nobre

Instituto Federal de Goiás (IFG)
Pós-Graduação em Inteligência Artificial Aplicada

Modelagem de Dados para IA • Aprendizagem de Máquina • Cloud
Computing

Professores

Prof. Dr. Sirlon Diniz
Prof. Dr. Raphael Gomes
Prof. Dr. Otávio Calaça

O Problema e Foco da Apresentação

Contexto e Decisão Apoiada

Domínio: Aplicativos de saúde e fitness que utilizam sensores de smartwatch (acelerômetro e giroscópio de pulso).

A Decisão: Classificar automaticamente qual das 18 atividades do protocolo WISDM o usuário está realizando a cada janela de 10 segundos.

Impacto: Alimenta contadores de atividade e alertas de sedentarismo sem depender de registros manuais do usuário.



Fontes de Dados: O Dataset WISDM

Originário da UCI Machine Learning Repository, o dataset chega em duas naturezas distintas:

- **WISDM Smartphone and Smartwatch Activity and Biometrics Dataset (UCI).**
- **Chega em duas naturezas: arff_files/ (estruturado, 200 arquivos) e raw/ (semiestruturado, 204 arquivos: sinais brutos de 51 sujeitos).**
- **Decisão: o pipeline recalcula os atributos a partir do sinal bruto (raw/), não reaproveita o ARFF pronto.**



Acelerômetro

Medição de força linear nos eixos X, Y e Z.



Giroscópio

Taxa de rotação angular nos eixos X, Y e Z.

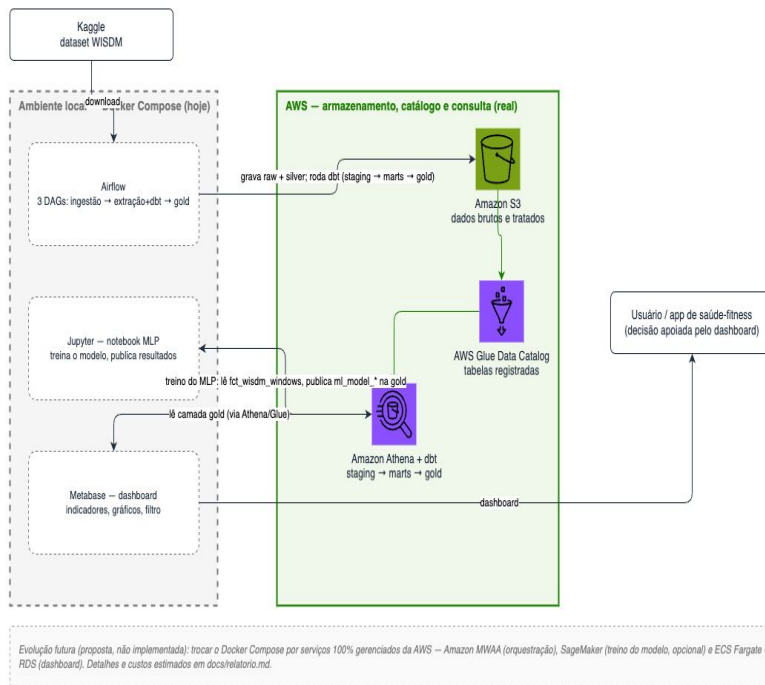
Arquitetura em Camadas

Adoção do padrão Medalhão (**Bronze, Prata, Ouro**) integrando recursos locais e serviços gerenciados.

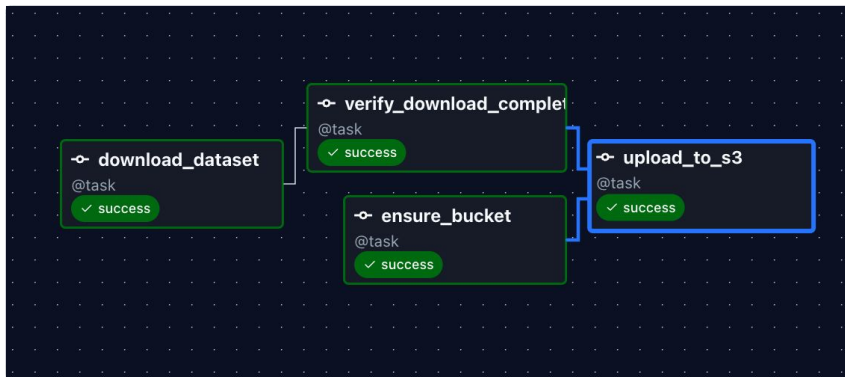
- **Cloud Computing (AWS):** Armazenamento (Amazon S3), Catálogo (AWS Glue) e Motor de Consulta Analítica (Amazon Athena).
- **Infraestrutura como Código:** Todo o ambiente AWS é provisionado via CloudFormation antes da execução.
- **Local (Docker):** Orquestração (Airflow), Treinamento (Jupyter) e Apresentação (Metabase) restritos ao laboratório.

Arquitetura real do pipeline WISDM – o que está implementado hoje

Orquestração e apresentação rodam localmente via Docker Compose (Airflow, Jupyter, Metabase); armazenamento, catálogo e consulta já são AWS real (S3, Glue Data Catalog, Athena – dbt roda em cima do Athena). A evolução para serviços 100% gerenciados é só uma nota ao final, não o foco deste diagrama.

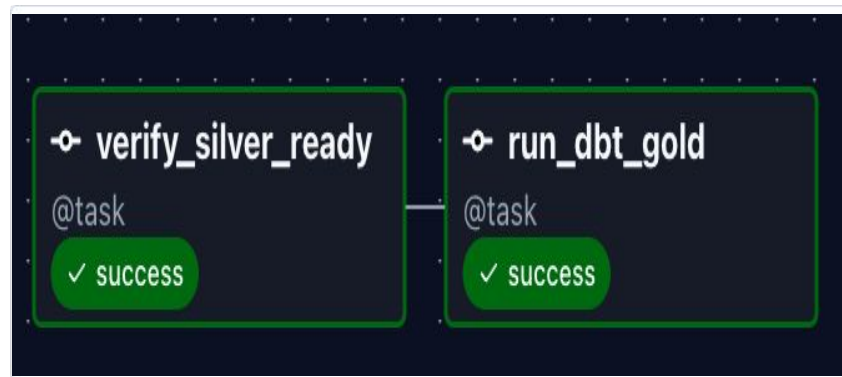


Pipeline de Ingestão e Processamento (Airflow)



1. Camada Bronze

Download do dataset. A DAG aplica validação de completude (51 sujeitos) antes do upload para o S3, prevenindo falhas silenciosas.



2. Encadeamento por Dados

Decisão Crítica: As DAGs não são agendadas por horário, mas via [Airflow Assets](#) (dependência de dados). A Prata aguarda a Bronze; a Ouro aguarda a Prata, garantindo rígida rastreabilidade.

DBT: staging, marts e gold

A modelagem tabular foi desenvolvida no dbt, executada diretamente sobre o Amazon Athena através do adapter dbt-athena, consolidando a união entre Modelagem de Dados e Cloud Computing.



Staging

Renomeação de atributos e criação de chaves substitutas granulares (window_uid) para cada janela de sinal.



Marts (Dimensional)

Aplicação da modelagem de Kimball.
Tabela fato central fct_wisdm_windows no grão de janela, base para o modelo de Machine Learning.



Gold & Testes

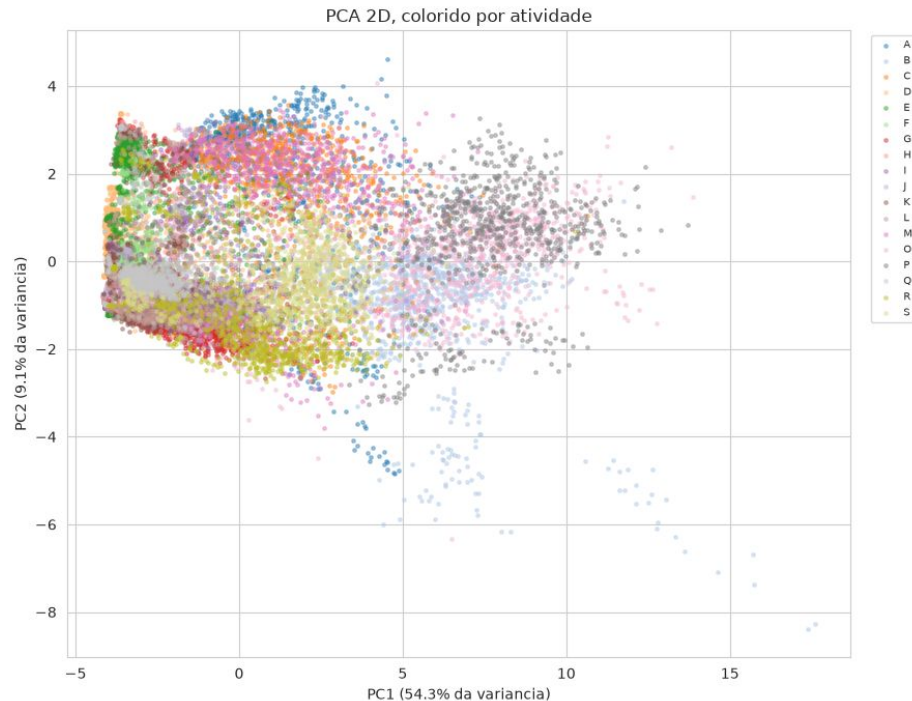
Agregados de negócio e KPIs. O pipeline conta com **23 testes rigorosos** (unicidade, nulos, validação de 18 rótulos).

Análise Exploratória

Classificação Multiclasse Supervisionada

- **Alvo:** 18 classes (balanceadas).
- **Features:** 24 atributos estatísticos por janela.
- **Rigor Metodológico:** O atributo subject_id foi explicitamente removido para evitar vazamento (memorização do indivíduo em vez do movimento).

A partir da fct_wisdm_windows, a Análise de Componentes Principais (PCA) 2D ao lado, observa-se que atividades de movimento amplo (correr, caminhar) formam clusters isolados, enquanto atividades estacionárias sobrepõem-se fortemente.



Preparação e split de dados

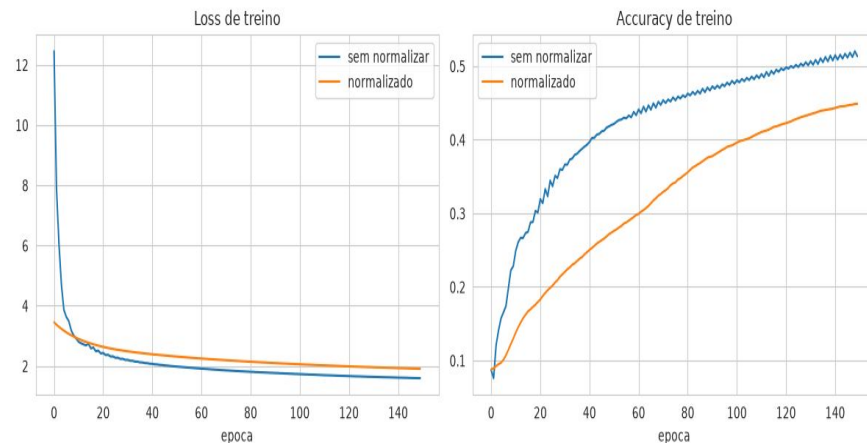
1. Split Estratégico por Sujeito

A divisão (35 Treino / 8 Validação / 8 Teste) foi realizada **estritamente por indivíduo**. Um split aleatório por linha vazaria padrões biomecânicos específicos do sujeito, inflando as métricas.

2. Normalização Z-Score

Necessária devido às diferentes escalas físicas (m/s^2 vs rad/s) e para a regularização L2. A estatística foi ajustada exclusivamente no conjunto de treino.

Curiosidade: No teste rápido (gráfico), o dado bruto convergiu mais rápido, evidenciando que normalizar nem sempre acelera o gradiente, embora seja vital para regularização equitativa.



MLP Implementado do Zero

O modelo foi implementado em NumPy puro, expondo a matemática subjacente da rede neural sem a abstração do scikit-learn.

Arquitetura: Camada de Entrada (24) → Camada Oculta (ReLU) → Camada de Saída (18, Softmax).

```
def __init__(self, n_input, n_hidden, n_output,
              learning_rate=0.05, momentum=0.9,
              l2_lambda=0.0, seed=42):
    rng = np.random.default_rng(seed)
    self.W1 = rng.normal(0, np.sqrt(2.0 / n_input),
                          size=(n_input, n_hidden))
    self.b1 = np.zeros((1, n_hidden))
    self.W2 = rng.normal(0, np.sqrt(2.0 / n_hidden),
                          size=(n_hidden, n_output))
    self.b2 = np.zeros((1, n_output))
```

MLP: Forward Pass e Retropropagação

```
def forward(self, X: np.ndarray) -> dict:
    Z1 = X @ self.W1 + self.b1
    A1 = relu(Z1)
    Z2 = A1 @ self.W2 + self.b2
    A2 = softmax(Z2)
    return {"Z1": Z1, "A1": A1, "Z2": Z2, "A2": A2}

def backward(self, X: np.ndarray, y_onehot: np.ndarray,
cache: dict) -> dict:
    n = X.shape[0]
    dZ2 = (cache["A2"] - y_onehot) / n
    dW2 = cache["A1"].T @ dZ2 + self.l2_lambda * self.W2
    db2 = dZ2.sum(axis=0, keepdims=True)
    dA1 = dZ2 @ self.W2.T
    dZ1 = dA1 * relu_derivative(cache["Z1"])
    dW1 = X.T @ dZ1 + self.l2_lambda * self.W1
    db1 = dZ1.sum(axis=0, keepdims=True)
    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}
```

O backward pass computa os gradientes retroativamente. A combinação analítica da ativação *Softmax* com a função de custo *Entropia Cruzada* simplifica notavelmente a derivada do erro da camada final para uma subtração direta.

$$\frac{\partial L}{\partial Z_2} = \frac{1}{N} (A_2 - Y)$$

A penalidade da Regularização L2 é somada aos pesos para mitigar o *overfitting*.



SGD com Momentum

A atualização dos pesos utiliza momento (fator 0.9) para acumular velocidade direcional, amortecendo oscilações e transpondo mínimos locais rasos com maior eficiência térmica no espaço de parâmetros.



Parada Antecipada

Monitoramento contínuo da validação (patience = 25 épocas). Ao estagnar, o treino é abortado e os pesos são **revertidos** para a época de menor erro, prevenindo degradação por *overfitting* severo.



Busca de Hiperparâmetros

Busca exaustiva em grade (18 configurações). A otimização baseou-se estritamente no **F1-Macro de Validação**, blindando o conjunto de teste para a inferência final incondicionada.

Resultados no Conjunto de Teste

Comparação de métricas com sujeitos inéditos, comprovando o aprendizado generalizado.

Modelo	Acurácia	F1 (macro)
Baseline Majoritário (Sorteio Viesado)	0.056	0.006
✓MLP Implementado do Zero (NumPy)	0.676	0.668
MLP Scikit-learn (Validação)	0.678	0.670

A divergência de apenas ~0.2% entre a implementação manual e o pacote consolidado confirma a integridade matemática do código customizado desenvolvido.

27.5%

de todo o erro de classificação do modelo

O Fator Limitante: Física, não Algoritmo

O erro total do modelo não é distribuído de forma homogênea. Ele se concentra em um cluster comportamental específico:

Comer/Beber sentado (sopa, batata frita, macarrão, bebida, sanduíche).

Mais de um quarto dos erros ocorre na confusão *interna* destas 5 classes.

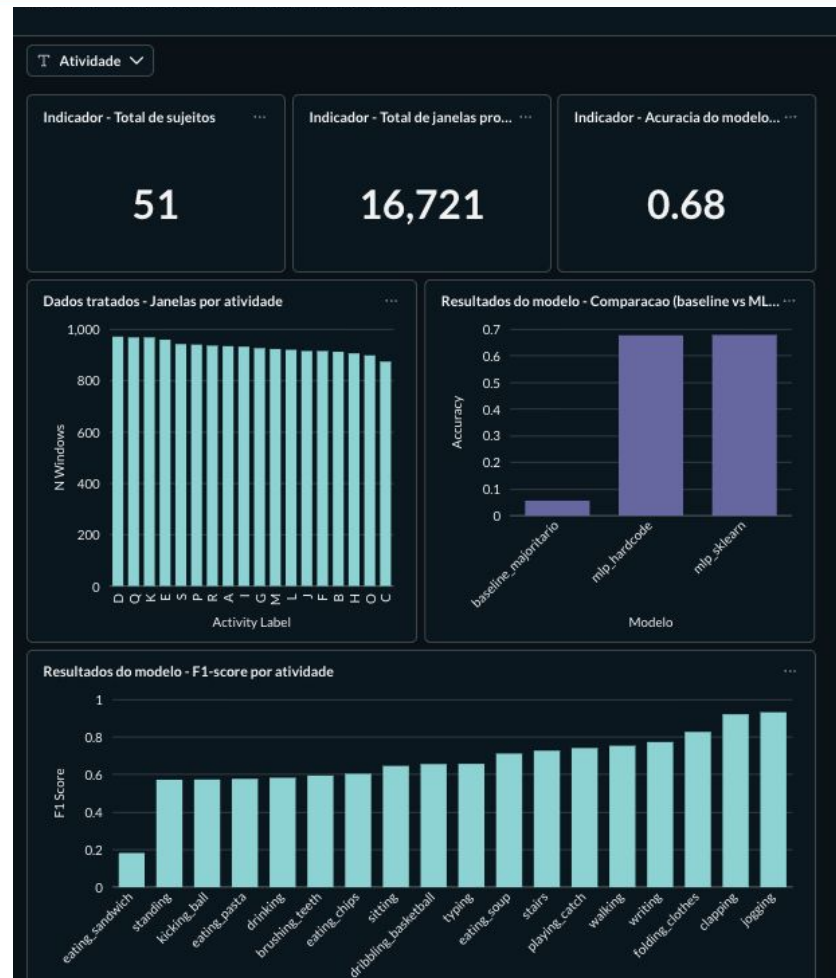
Validação da Hipótese: Validação da Hipótese: Ao aglutinar estas 5 classes em um único rótulo macro, a acurácia global saltou de 0.676 para 0.777. Conclusão: a instrumentação isolada no pulso carece de dados dinâmicos para diferenciar o objeto manuseado.

Painel de Decisão (Metabase)

Monitoramento Contínuo

Os resultados convergem em um painel Metabase conectado via Amazon Athena diretamente à **Camada Ouro** do Data Lake.

- **KPIs Centrais:** Volume de sujeitos (51), instâncias analíticas, e eficácia sintética (Acurácia de 0.68).
- **Recorte Analítico:** Filtros interativos permitem dissecar o desempenho por atividade em tempo real.
- Visualização direta do déficit do F1 nas atividades de alimentação.



Conclusão

- **Limitações:** só dispositivo watch; 51 sujeitos; grupo comer/beber não distinguível só com sensor de pulso.
- **Próximos passos:** migrar para arquitetura 100% AWS gerenciada (MWAA, SageMaker, ECS Fargate com RDS); processar também os dados de phone; classificador hierárquico para o grupo comer/beber.

Referências

- AMAZON WEB SERVICES. Amazon Web Services (AWS). 2026. Disponível em: <https://aws.amazon.com/>. Acesso em: 13 ago 2026.
- APACHE SOFTWARE FOUNDATION. Apache Airflow Documentation. 2024. Disponível em: <https://airflow.apache.org/docs/>. Acesso em: 12 ago 2026.
- DBT LABS. dbt Documentation. 2024. Disponível em: <https://docs.getdbt.com/>. Acesso em: 12 ago 2026.
- HARRIS, Charles R. et al. Array programming with NumPy. Nature, v. 585, n. 7825, p. 357–362, 2020.
- KIMBALL, Ralph; ROSS, Margy. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. 3. ed. Indianapolis: Wiley, 2013.
- METABASE. Metabase. 2026. Disponível em: <https://www.metabase.com/>. Acesso em: 13 ago 2026.
- NOBRE, Igor Coutinho. Projeto Integrado – Segundo Bimestre. 2026. Repositório GitHub. Disponível em: <https://github.com/coutinhonobre/Projeto-Integrado-Segundo-Bimestre>. Acesso em: 13 ago 2026.
- WEISS, Gary. WISDM Smartphone and Smartwatch Activity and Biometrics Dataset. [S.I.]: UCI Machine Learning Repository, 2019. Dataset. Acesso em: 12 ago 2026.